

Real-Time Chat and User Tracking System

A Real-Time Communication System Using Node.js, Socket.IO, and MySQL

Submitted By

Name: Vivek Gupta

College: Arka Jain University, Jamshedpur

Department: Computer Science and Engineering

Academic Session: 2025–2026

Project Type: Networking Based Web Application

Abstract

The Real-Time Chat and User Tracking System is a browser-based real-time messaging platform designed for communication within a local network environment. The primary objective of the system is to provide instant communication between users using web technologies and socket-based communication.

The application is developed using Node.js and Express.js on the backend, Socket.IO for real-time bidirectional communication, and MySQL for storing user and message data. The frontend interface is built using HTML, CSS, and JavaScript with a modern and responsive design.

The system supports features such as user joining, online user tracking, live messaging, typing indicators, and database logging. Messages are transmitted instantly without requiring page refresh, which improves user experience and communication efficiency.

The project demonstrates the practical implementation of real-time networking concepts and web socket technology in modern web applications.

Chapter 1: Introduction

1.1 Introduction to the Project

Communication systems are widely used in modern digital environments for instant interaction and information sharing. Real-time messaging applications help users communicate quickly and efficiently without delays. These systems are commonly used in educational institutions, offices, and local networks.

This project focuses on developing a Real-Time Chat and User Tracking System using Node.js, Socket.IO, and MySQL. The application allows users to send and receive messages instantly through a web-based interface. It also supports online user tracking and database storage using modern web technologies.

1.2 Objectives of the Project

The main objectives of the project are:

- .To develop a real-time messaging system.**
- .To allow users to communicate instantly over a local network.**
- .To maintain records of users and messages in a database.**
- .To display online users dynamically.**
- .To create a responsive and user-friendly interface.**

1.3 Scope of the Project

The project can be implemented in:

- Computer laboratories
- Educational institutions
- Small office networks
- Local communication systems
- Internal organization communication

The system can further be enhanced by integrating advanced features such as private chat rooms, media sharing, authentication systems, and cloud deployment.

1.4 Advantages of the System

- Fast real-time communication
- Lightweight and efficient architecture
- Easy to use interface
- Low resource consumption
- Database integration for message storage
- Dynamic online user tracking

Chapter 2: System Analysis

2.1 Existing System

Traditional communication systems often depend on external applications or platforms that may not be suitable for local communication environments. Many existing systems require complex installation and authentication processes.

These systems may also lack proper customization and local control.

2.2 Proposed System

The proposed system is a browser-based LAN chat application that enables users to exchange messages instantly through real-time socket communication.

The application uses:

- Node.js for backend execution
- Express.js for server management
- Socket.IO for real-time communication
- MySQL for database management
- HTML, CSS, and JavaScript for frontend development

2.3 Feasibility Study

Technical Feasibility

The project is technically feasible because the required technologies are open-source, lightweight, and widely supported.

Economic Feasibility

The system is economically feasible because it does not require expensive hardware or paid software. **Operational**

Feasibility

The application is simple and user-friendly, making it easy for users to operate.

2.4 Software Requirements

- Operating System: Windows/Linux/macOS
- Node.js
- Express.js
- Socket.IO
- MySQL Server
- Visual Studio Code
- Web Browser

Chapter 3: System Design

3.1 System Architecture

The application follows a client-server architecture.

Client Side

The client interface is responsible for:

- User interaction
- Sending messages
- Displaying messages
- Displaying online users
- Showing typing indicators

Server Side

The server handles:

- Socket connections
- Event broadcasting
- User management
- Database communication
- Message storage

3.2 Database Design

The system uses MySQL database with two primary tables:

Users Table

Field Name	Description
id	Unique User ID
socket_id	Socket Connection ID
username	Username
connected_at	Connection Time
disconnected_at	Disconnection Time
status	User Status

Messages Table

id	Message ID
user_id	User Reference
message	Message Content
sent_at	Message Timestamp
Field Name	Description

3.3 User Interface Design

The frontend interface contains:

- Login/Join Section
- Chat Window
- Online Users Panel
- Typing Indicator
- Message Composer
- Responsive Layout

The interface uses modern styling techniques with smooth user interaction.

Chapter 4: Implementation

4.1 Technologies Used

Node.js

Node.js is used as the runtime environment for executing backend JavaScript code.

Express.js

Express.js is used for server routing and HTTP handling.

Socket.IO

Socket.IO enables real-time bidirectional communication between clients and the server.

MySQL

MySQL is used for storing user information and chat messages.

HTML, CSS, and JavaScript

These technologies are used for frontend interface design and functionality.

4.2 Working Procedure

1. User opens the application.
2. User enters a username.
3. Socket connection is established.
4. User joins the chat room.
5. Messages are transmitted instantly.
6. Messages are stored in the database.
7. Online user list updates dynamically.
8. User disconnect events are recorded.

4.3 Event Flow

The system handles the following events:

- Join Event/Message Event
- Typing Event/Disconnect Event

These events are processed in real time using Socket.IO.

Chapter 5: Testing

5.1 Testing Objectives

The testing phase ensures that the application performs correctly and all functionalities work efficiently.

5.2 Test Cases

Test Case 1: User Join

Input	Expected Output	Result
Username	User joins successfully	Passed

Test Case 2: Send Message

Input	Expected Output	Result
Text Message	Message delivered instantly	Passed

Test Case 3: Online User Display

Input	Expected Output	Result
User Connection	User appears online	Passed

Test Case 4: Database Storage

Input	Expected Output	Result
Message Data	Stored in MySQL	Passed

Test Case 5: Disconnect Handling

	Expected Output	Result
Browser Close	User marked offline	Passed

Input

5.3 Testing Result

All functionalities were tested successfully. The application worked efficiently without major issues.

Chapter 6: Results and Discussion

The developed Real-Time Chat and User Tracking System successfully achieved all intended objectives.

The application provides:

- .Real-time communication
- .Instant message delivery
- .Online user monitoring
- .Database storage functionality
- .Responsive user interface
- .Efficient socket communication

The system demonstrated stable performance and smooth interaction during testing.

Chapter 7: Conclusion

The Real-Time Chat and User Tracking System was successfully developed using Node.js, Express.js, Socket.IO, MySQL, HTML, CSS, and JavaScript.

The project helped in understanding:

- Real-time networking concepts
- Socket communication
- Database connectivity
- Client-server architecture
- Event-driven programming
- Modern web application development

The system fulfills all major objectives and provides an efficient communication platform for local network environments.

Chapter 8: Future Enhancements

The system can be improved further by adding:

- User authentication system
- Private messaging
- Voice and video calling
- File and media sharing
- Group chat support
- Notification system
- Cloud deployment
- Mobile application support
- End-to-end encryption

References

1. Node.js Official Documentation
2. Socket.IO Official Documentation
3. MySQL Documentation
4. Express.js Documentation
5. HTML, CSS, and JavaScript Tutorials
6. Web Socket Programming References
7. Networking Concepts and Web Application Resources

Appendix

SQL Query for Users Table

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    socket_id VARCHAR(100) NOT NULL,  
    username VARCHAR(50) NOT NULL,  
    connected_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    disconnected_at DATETIME NULL,  
    status ENUM('online', 'offline') DEFAULT 'online'  
);
```

SQL Query for Messages Table

```
CREATE TABLE messages (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    message TEXT NOT NULL,  
    sent_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Final Outcome

The project was completed successfully and demonstrates the implementation of a professional real-time LAN chat system using modern web technologies. The application provides efficient communication, proper database management, and a responsive user interface suitable for networking-based academic projects.